

SparseSAM: Structured Sparsification of Activations in Segment Anything Models

Hoai-Chau Tran^{1,3}, Chi H. Nguyen^{2,3}, Duy M. H. Nguyen^{4,5,6},
Mathias Niepert^{5,6}, Fan Lai¹, Khoa D. Doan^{2,3}.

¹ University of Illinois at Urbana-Champaign

² College of Engineering & Computer Science, VinUniversity,

³ VinUni-Illinois Smart Health Center, VinUniversity, ⁴ DFKI

⁵Max Planck Research School for Intelligent Systems (IMPRS-IS), ⁶University of Stuttgart,
{chauht2}@illinois.edu

Abstract

The Segment Anything Model (SAM) achieves strong open-vocabulary segmentation, but its ViT-based image encoders dominate inference latency and memory. Existing activation compression methods, such as token merging, reduce the token length to process, yet introduce non-trivial runtime overhead and encounter catastrophic quality drop under high compression. Other methods applying Sparse Attention focus on attention alone, leaving the MLP fully dense and capping achievable speedup. We propose *SparseSAM*, a (i) *training-free structured sparsification* framework that jointly accelerates attention and MLP layers while preserving token identity. SparseSAM introduces (ii) *Stripe-Sort Attention*, which uses a deterministic Z-order permutation to transform dense attention into static hardware-friendly sparse patterns, eliminating dynamic masking overhead. SparseSAM further introduces a (iii) *Residual-Consistency MLP* that routes only informative tokens through the MLP while propagating remaining tokens through the residual pathway. Across four segmentation benchmarks, SparseSAM loses only 0.004 mIoU at a 0.4 density and 0.021 mIoU at 0.3, a 2.10× reduction in accuracy loss versus token merging advances, while achieving 2x faster inference and 2.8× memory reduction.

1 Introduction

SAMs have demonstrated strong performance in image segmentation and are widely adopted across downstream tasks, driven in large part by their strong zero-shot generalization capabilities [15, 19, 18, 17]. Unlike large language models (LLMs), SAMs follow a modular architecture consisting of an image encoder, a prompt encoder, and a lightweight mask decoder. The image encoder overwhelmingly dominates both computation and model parameters (by over 99%). Serving SAM in latency-sensitive or resource-constrained environments, such as large-scale serving systems or edge devices, remains prohibitively expensive.

To meet the pressing demand for efficient inference, recent advances have explored replacing SAM’s image encoder with more compact architectures [33, 38, 30, 37]. While reducing inference cost, they typically require training an entirely new model, incurring substantial training and data collection overheads, yet often introduce non-trivial accuracy degradation. An alternative direction focuses on training-free, post-training optimization, such as exploiting attention sparsity through customized kernels originally designed for LLMs and diffusion models [11, 29, 28]. By skipping less important attention blocks, these methods sidestep dense computation but encounter an architectural mismatch due to SAM’s multi-scale design: SAM partitions each image into a large number of local regions, producing hundreds of attention heads with fewer than 200 tokens each, and at this granularity, the overhead of computing dynamic sparsity masks frequently offsets any theoretical speedups.

Beyond attention, the MLP blocks contribute substantially to the encoder latency, necessitating holistic compression strategies. Existing efforts, however, remain insufficient. MLP pruning advances [3] require expensive retraining or knowledge distillation. Post-training quantization [20, 27, 23, 36] targets low-bit representations (e.g., W4A4) but rarely delivers measurable GPU speedups due to SAM’s relatively small matrix dimensions. Similarly, activation compression via token merging [1, 25, 26] is also ill-suited for this SAM. Because segmentation requires the model to preserve the full token set at the output, every merge operation must be followed by a corresponding unmerge step to restore the original spatial resolution. This per-layer administrative overhead ultimately outweighs the computational savings, frequently resulting in higher overall latency and degraded quality (Fig. 8).

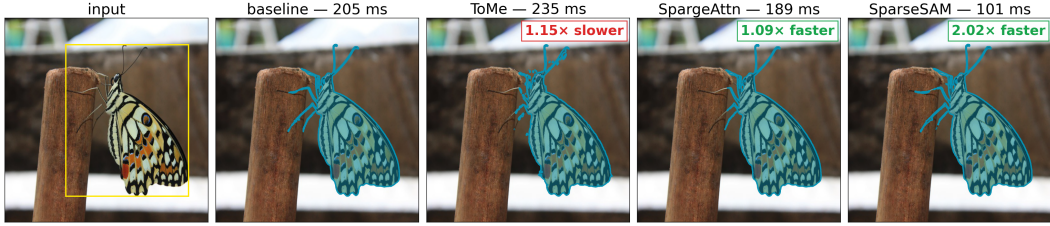


Figure 1: SparseSAM enables $2.02\times$ faster SAM inference while preserving quality.

These limitations point to a key gap: *efficient, training-free methods that jointly reduce computation in both attention and MLP layers, while preserving token-level fidelity required for segmentation*. We introduce *SparseSAM*, a training-free approach designed to leverage the inherent sparsity exposed by the activations in both the attention and MLP layers of SAM. We propose two complementary techniques that improve encoder throughput while preserving segmentation quality.

- **Stripe-Sort Attention.** We introduce a new structured sparse attention mechanism based on a deterministic Z-order (Morton) permutation [21, 7, 24]. This reordering induces spatial locality in token indices, enabling a static, block-structured sparsity pattern implemented via a custom CUDA kernel. The resulting pattern consists of a banded diagonal capturing local interactions, augmented with a small global keep set for long-range dependencies. Notably, this design avoids dynamic mask construction and preserves the full key/value set, ensuring both efficiency and quality (Fig. 5 and Fig. 6).
- **Residual-Consistency MLP.** We propose a token routing mechanism that applies the MLP only to a top- K subset of tokens selected based on their importance (i.e., the high-rank prefix of the sorted z-group tokens), while allowing the remaining tokens to bypass the MLP through the residual connection. This design retains information flow without incurring the full cost of dense MLP computation (Fig. 7a).

Both techniques utilize a **static, one-shot permutation set** that is computed once and reused across all model layers, ensuring that per-layer overhead remains negligible. Our evaluations across five distinct SAM checkpoints and five benchmarks demonstrate that SparseSAM achieves an average $2.0\times$ inference speedup. Even at high-density compression rates, the method maintains 50% sparsity with a minimal segmentation accuracy drop ($<1\%$ IoU loss), as illustrated in Figure 8.

2 Background and Related Work

SAM Architecture. SAM adopts a hierarchical Vision Transformer (ViT) encoder composed of both local and global attention blocks. Local blocks partition the image into non-overlapping windows and apply self-attention independently within each window, while global blocks perform dense attention over the entire token sequence to capture long-range dependencies.

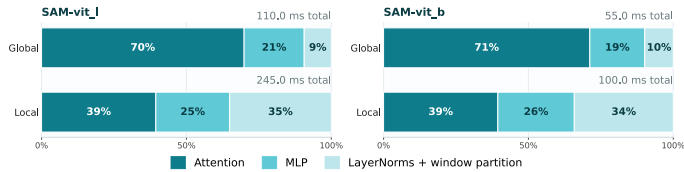


Figure 2: SAM latency profiling.

As shown in Fig. 2, the primary computational bottleneck in global attention blocks is the attention operator, which can account for up to 70% of runtime. In contrast, *local blocks* exhibit a *dual bottleneck*, where both attention and MLP layers contribute significantly to latency. This distinction is important because SAM architectures contain substantially more local blocks than global blocks. For example, SAM-L contains 20 local blocks but only 4 global blocks. Consequently, optimizing attention alone provides limited end-to-end acceleration; practical speedups require jointly reducing the cost of both attention and MLP computation.

Sparse Attention Approaches. Recent advancements have leveraged sparsity to mitigate the quadratic cost of Transformer attention. SparseAttention [35] employs a two-stage online filtering mechanism, while PISA [14] utilizes block-wise Taylor expansions to approximate non-critical regions. Additionally, the Sparse Video Gen series [28, 31] introduces semantic-aware permutations to cluster salient tokens.

While effective for long-context Transformers and diffusion models, where *sequence lengths reach tens of thousands*, these approaches *encounter a "complexity wall"* when applied to the SAM due to its architecture, which primarily relies on local-window attention with fewer than 200 tokens per head. In this regime, the computational overhead of dynamic mask construction and online importance estimation often outweighs the savings from skipped operations. Furthermore, existing sparse attention methods fail to address the substantial MLP costs incurred by local blocks, thereby capping potential end-to-end acceleration (Fig. 8). Unlike these dynamic approaches, our method leverages a **deterministic Z-order permutation** to induce a static, hardware-efficient sparsity pattern that accelerates both attention and MLP components with zero runtime search overhead.

Activation Compression Approaches. A complementary research direction focuses on reducing activation density to jointly accelerate attention and MLP layers. Weight-activation quantization [20, 27, 23, 36] reduces theoretical FLOPs and memory footprints; however, modern INT4 Tensor Core kernels require large contraction dimensions to saturate GPU throughput [13]. Since SAM operates on relatively small matrix tiles (typically 64 to 1024), the fixed overheads of quantization, including scaling factor computation and kernel launch latency; thus often negate the gains from reduced precision.

Alternatively, token merging [1, 25, 26] reduces the effective sequence length by coalescing redundant tokens. While successful in global classification tasks, this approach is fundamentally ill-suited for high-fidelity segmentation, where the decoder demands full spatial resolution. Restoring this resolution necessitates frequent per-layer merge-unmerge cycles, introducing scatter-gather overheads [22] that typically exceed the computational savings. Furthermore, aggressive merging erodes fine-grained token distinctiveness, leading to a precipitous decline in mask quality at higher compression rates. In contrast, our approach utilizes a **structure-preserving Z-order sparsity** that retains the original token grid, bypassing both the latency of quantization and the information loss of merging.

3 Observations and Motivation

Since SAM is exceptionally sensitive to overheads, we require an approach that minimizes per-layer administrative costs while maintaining high-fidelity feature representations.

Rather than dynamically identifying sparse regions, we explore an alternative perspective: *can we restructure the token layout itself such that efficient sparsity patterns emerge deterministically?* We next introduce the key insights that motivate our work.

Observation 1: Structured token permutation induces reusable sparse attention patterns. Our design is motivated by prior activation compression studies [25, 26], which show that preserving spatial diversity during token compression is substantially more important than preserving contiguous foreground regions. Intuitively, segmentation quality depends heavily on maintaining broad spatial coverage across the image. Therefore, we seek a way to

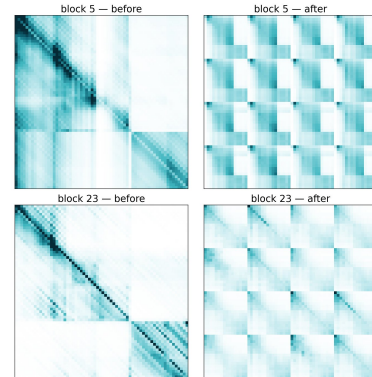


Figure 3: **Permuted attention visualization.** Comparison between the original SAM attention pattern (left) and our Z-order permuted pattern (right). The permutation induces a banded diagonal structure, allowing for significant hardware acceleration with minimal loss in spatial information.

permute the $\mathbf{Q}$ and $\mathbf{K}$ matrices such that the resulting attention computation effectively performs spatial downsampling while preserving the global distribution of tokens within each sub-attention map.

To this end, we introduce the **stripe-sort attention kernel**, which applies a parameter-free, deterministic permutation. This reordering incurs minimal overhead and can be readily fused into existing attention kernels. As illustrated in Figure 3, the permutation reorganizes tokens such that the original large attention map is decomposed into four smaller sub-attention maps, each exhibiting a similar attention pattern. In effect, the operation produces multiple phase-shifted views of the full attention, while maintaining broad spatial coverage. Further details are provided in Section 4.

Observation 2: SAM’s decoder depends more on inter-region contrast than precise per-token representations.

We further observe that SAM exhibits substantial representational redundancy, stemming from how the mask decoder utilizes encoder outputs. To investigate this, at each block we perform k -means clustering on the attention outputs and replace the encoder’s 4096 tokens after the MLP update with their nearest cluster centroids. Remarkably, even with $k = 128$ (approximately 3% of the original token counts), the decoder retains 93% of the baseline mIoU on COIFT (Fig. 4), with performance saturating near the baseline at $k = 256$. These results suggest that tokens within the same cluster are largely interchangeable from the decoder’s perspective. What matters is the *contrast between clusters*, which enables the model to distinguish different objects, rather than the precise representation of each individual patch.

Observation 3: Which tokens actually require expensive MLP updates?

We analyze how MLP layers modify token representations and find that SAM’s MLP exhibits a *strong inductive bias toward separating distinct image regions*: a large fraction of background tokens remain close to the residual branch, while semantically rich foreground tokens undergo significantly larger updates (Fig. 7b). To better quantify this behavior, in Table 1 we measure the correlation between the MLP update magnitude, $\|\Delta_{MLP}\|_2$, and token dissimilarity (measured via cosine distance). We observe that tokens that are more distinct in feature space tend to receive larger updates, with a strong positive correlation ($\rho \approx 0.6$ – 0.7).

This redundancy motivates our second contribution, *residual-consistency MLP*, which prioritizes MLP updates for semantically important tokens while maintaining lightweight residual propagation for the remaining tokens.

4 Methods

4.1 Stripe-Sort Attention

We first introduce *Stripe-Sort Attention*, a structured sparse attention mechanism that reorganizes token layouts to expose deterministic sparsity patterns amenable to efficient GPU execution. Specifically, we use a deterministic, parameter-free permutation that decomposes the $N \times N$ attention matrix into a 4×4 mosaic of $(N/4) \times (N/4)$ sub-maps. Since the permutation depends only on spatial position, it can be precomputed once and statically compiled into the attention kernel with no runtime overhead. Importantly, the four resulting token subsets are phase-shifted, half-resolution views of the original image, where each subset remains spatially distributed across the entire input rather than localized to a single region. Consequently, each sub-map preserves global context while operating on only a quarter of the tokens.

Gradient-Based Ordering. Given an input feature map $\mathbf{X} \in \mathbb{R}^{H \times W \times D}$, our goal is to construct a one-dimensional token ordering that places high-information regions early in the sequence while spreading them evenly across spatial positions. The first property ensures that when the sequence

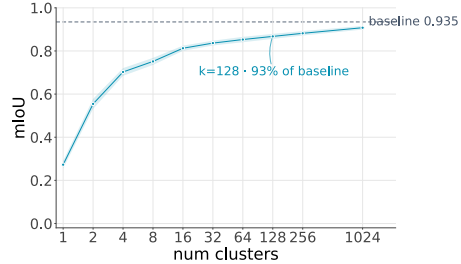


Figure 4: K-means clustering replacement.

	Layer 1	Layer 6	Layer 16	Layer 22
ρ	0.64	0.62	0.75	0.78

Table 1: Layer-wise correlation between token dissimilarity and the norm of $\|\Delta_{MLP}\|_2^2$ for each tokens.

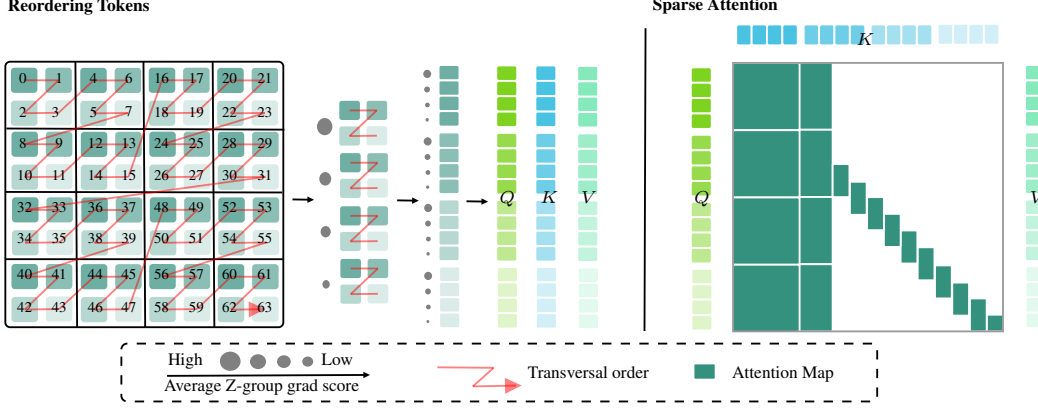


Figure 5: We employ a deterministic **Z-order (Morton) traversal** to linearize 2D spatial tokens into a 1D sequence. This traversal naturally preserves spatial locality by grouping neighboring pixels into localized "Z-groups." Such grouping enables the efficient computation of gradient-based importance scores within local windows (Eq. 1), which are then used to rank and route tokens. The resulting permuted sequence transforms the attention map into a hardware-friendly block-banded structure, consisting of a dense global prefix for high-saliency tokens and a banded diagonal for local spatial interactions.

is later truncated or sparsified, the most informative tokens are retained, while the second property ensures that the retained tokens collectively cover the full spatial extent of the image rather than concentrating in a single region.

To achieve both properties, we use local image gradients as a lightweight proxy for information density. Intuitively, regions with large gradient magnitude correspond to object boundaries, texture, and visually salient structures, while regions with low gradient magnitude correspond to smooth or homogeneous areas that contribute relatively little to dense prediction. We measure local gradient magnitude using the Sobel operator [9], a classical edge-detection filter that approximates the spatial derivatives of an image through a pair of fixed 3×3 convolutional kernels. Specifically, the gradient magnitude at each spatial position is

$$\mathbf{M}[i, j] = \sqrt{(\mathbf{S}_x * \mathbf{X})^2[i, j] + (\mathbf{S}_y * \mathbf{X})^2[i, j]}, \quad (1)$$

where $\mathbf{S}_x$ and $\mathbf{S}_y$ denote the horizontal and vertical Sobel kernels, and $*$ represents channel-wise 2D convolution followed by summation across the D channels. The resulting map $\mathbf{M} \in \mathbb{R}^{H \times W}$ is a single-channel saliency map, where high $\mathbf{M}$ values correspond to object boundaries and low values correspond to smooth regions. This computation is fully deterministic, parameter-free, and only performed once in the first SAM layer, introducing negligible overhead. Sorting the $N = HW$ spatial positions in descending order of $\mathbf{M}$ produces a permutation π , placing high-gradient regions at the front and low-gradient regions at the back.

Stripe-Sort Permutation. To prevent foreground regions from gathering at the start of the token set, we view π as a matrix $\mathbf{T} \in \mathbb{N}^{(N/G) \times G}$ with $\mathbf{T}[t, g] = \pi[t \cdot G + g]$ and define the final scan order as $\sigma = \text{flatten}(\mathbf{T}^\top)$.

This visits every G -th element of π before returning to the next offset, so each of the G resulting blocks of σ contains a uniformly subsampled mixture of tokens drawn from across the entire image. As demonstrated in Figure 6, applying this reordering to both queries and keys transforms global attention into a deterministic block-diagonal pattern that can be efficiently executed using static block-sparse kernels like FlashAttention. Our method avoids runtime mask generation, semantic clustering, and online scheduling overhead.

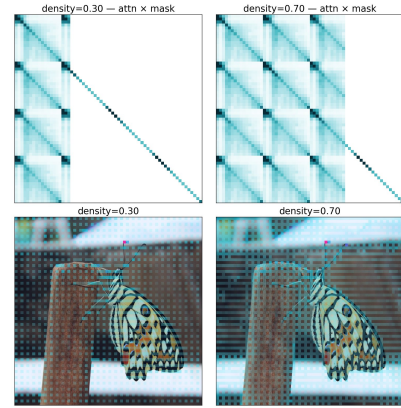


Figure 6: Effect of Stripe-Sort Attention with different sparsity ratios

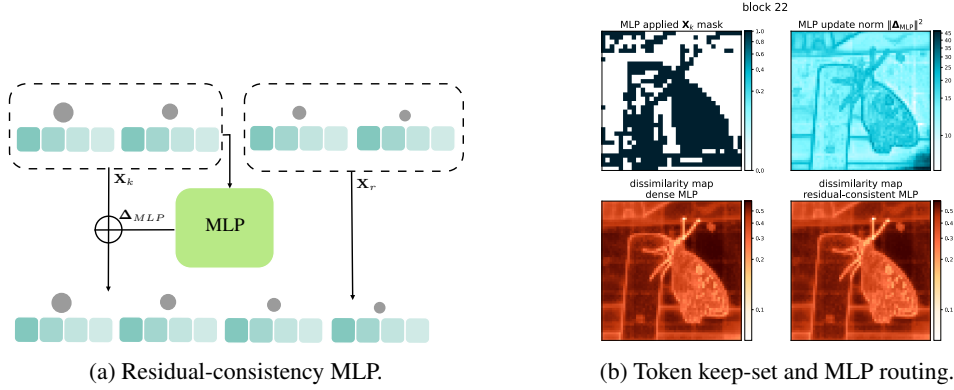


Figure 7: Visualization of the residual-consistency MLP. (a) The deterministic token order π is reused to partition tokens into a keep-set $\mathbf{X}_k$, updated by the MLP, and a residual set $\mathbf{X}_r$, which bypasses the MLP unchanged. (b) Our analysis shows that dense SAM implicitly focuses MLP updates on a small set of representative, highly dissimilar tokens. After applying the residual-consistency MLP, the inter-token similarity structure remains close to the dense baseline, preserving the representational geometry required by the segmentation decoder.

We implement Stripe-Sort Attention as a fully fused CUDA kernel to eliminate intermediate memory scattering and redundant kernel launches. In local layers, each sub-image contains at most 196 tokens, making standard FlashAttention [4, 5] and FlashInfer [32] kernels inefficient due to their fixed tile sizes (e.g., 128×128). We therefore design a custom CUDA kernel with flexible tiling, using 32×32 tiles for local layers and 128×128 tiles for global layers, while maintaining efficient `mma.sync.aligned.m16n16k16` tensor-core execution. The pseudo-code of the kernel is provided in Appendix A.1.

4.2 Residual-Consistency MLP

Our second contribution is motivated by the structure of MLP updates: the decoder depends more on inter-region contrast than precise per-token representations. Given input tokens $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N] \in \mathbb{R}^{N \times d}$, the MLP update for token i is $\Delta_i = \text{MLP}(\text{LN}(\mathbf{x}_i))$, where $\text{LN}(\cdot)$ denotes LayerNorm. We use the update magnitude $u_i = \|\Delta_i\|_2$ to quantify how strongly each token is modified.

As illustrated in Fig. 7b, the distribution of $\{u_i\}_{i=1}^N$ is highly skewed across encoder blocks: only a small subset of tokens receives large updates, while most remain close to the residual connection. Spatially, these high-update tokens correspond to regions with strong feature dissimilarity, such as object boundaries and salient textures, whereas smooth regions receive minimal updates. Consistent with this observation, Table 1 shows a strong correlation between token uniqueness and update magnitude u_i , suggesting that the dense MLP naturally performs an implicit form of routing.

$$\mathbf{y}_i = \begin{cases} \mathbf{x}_i + \Delta_i, & i \in \mathcal{K}, \\ \mathbf{x}_i + \epsilon_i, & i \notin \mathcal{K}, \end{cases}$$

where $\mathcal{K}$ denotes a small subset of representative tokens that receive meaningful updates and ϵ_i is typically much smaller than Δ_i .

Formulation. Motivated by this observation, we make the routing behavior explicit. Given we reuse the deterministic permutation order to reorder the tokens $\pi(\mathbf{X})$ and partition the sequence into a keep-set $\mathbf{X}_k = \{\mathbf{x}_i \mid i \in \mathcal{K}\}$ and a residual set $\mathbf{X}_r = \{\mathbf{x}_i \mid i \notin \mathcal{K}\}$. As shown in Fig. 7b(a), the MLP is applied only to the keep-set, while residual tokens bypass the MLP unchanged.

$$\mathbf{Y}_k = \mathbf{X}_k + \text{MLP}(\text{LN}(\mathbf{X}_k)), \quad \mathbf{Y}_r = \text{LN}(\mathbf{X}_r).$$

By routing only representative tokens through the MLP, our method preserves the feature geometry of the dense baseline while substantially reducing MLP computation. Despite its simplicity, this approach proves highly effective. As shown in Fig. 7b, the inter-token dissimilarity maps produced by the dense MLP and our residual-consistency MLP are nearly identical, indicating that the routed computation preserves the representational structure required by the segmentation decoder.

Table 2: Zero-shot segmentation on MS-COCO datasets using DINO as a bounding box detector generates a box prompt for the SAM model. Markers indicate compression type: ◦ attention-only and ● attention+MLP.

Method	SAM-B			SAM-L			SAM-H		
	mAP	Density	Speedup	mAP	Density	Speedup	mAP	Density	Speedup
Base	0.468	100%	× 1.00	0.495	100%	× 1.00	0.499	100%	× 1.00
◦ Sparge Attention	0.447	25%	× 1.25	0.461	25%	× 1.23	–	25%	–
	0.463	50%	× 1.24	0.491	50%	× 1.21	–	50%	–
◦ PieceWise Attention	0.455	25%	× 0.73	0.483	25%	× 0.68	0.486	25%	× 0.69
	0.465	50%	× 0.72	0.494	50%	× 0.67	0.498	50%	× 0.65
● ToMe	0.403	25%	× 0.83	0.425	25%	× 0.86	0.430	25%	× 0.80
	0.467	50%	× 0.74	0.494	50%	× 0.75	0.498	50%	× 0.64
● StructSAM	0.306	25%	× 0.86	0.318	25%	× 0.89	0.318	25%	× 0.94
	0.422	50%	× 0.71	0.445	50%	× 0.74	0.451	50%	× 0.79
◦ SparseSAM (Ours)	0.459	25%	× 1.63	0.487	25%	× 1.64	0.494	25%	× 1.28
	0.467	50%	× 1.55	0.494	50%	× 1.57	0.499	50%	× 1.22
● SparseSAM (Ours)	0.451	25%	× 2.15	0.468	25%	× 2.05	0.472	25%	× 1.59
	0.459	50%	× 1.89	0.482	50%	× 1.83	0.487	50%	× 1.40

5 Experiments

5.1 Experimental Setup

Models and Datasets. We implement SparseSAM on top of the official checkpoints of SAM-B, SAM-L, and SAM-H. Unless otherwise specified, all experiments and latency measurements are conducted on a single NVIDIA A100 GPU. We use HQ-44K [12] to evaluate high-quality fine-grained segmentation and MS-COCO [16] to evaluate zero-shot generalization on common object categories. For MS-COCO, we utilize DINO [34], H-DETR [10], and YOLOX [8] as the primary object detectors. We compare SparseSAM against two categories of baselines:

Attention Sparsification Baselines. We compare against (i) SPARGEATTENTION [35] and PIECEWISE SPARSE ATTENTION (PISA) [14], two state-of-the-art training-free sparse attention frameworks originally designed for large Transformer models. These methods dynamically identify important attention regions to reduce computation while approximately preserving dense attention behavior. To ensure fair comparison on SAM, we extend both implementations with fused relative positional encoding support, which is required by SAM’s ViT encoder but absent from their original implementations.

Activation Compression Methods Baselines. For both Attention and MLP compression, we compare against Token Merging (ToMe) [1] and its gradient-aware variant, StructSAM [22], designed for the SAM model. These methods represent the standard paradigm of using token reduction via feature similarity or gradient matching to lower activation density in Vision Transformers.

Metrics. We report segmentation quality using mean Intersection-over-Union (mIoU) and Boundary IoU (BIOU), which together capture both region-level accuracy and boundary fidelity. For system efficiency, we measure end-to-end latency, peak GPU memory usage, and throughput under varying density (i.e., $100\% - sparsity$) ratios. Unless otherwise specified, all latency results are averaged over multiple runs with synchronized CUDA timing.

5.2 Segmentation Results

5.2.1 Common Objects Segmentation

Table 2 reports zero-shot segmentation results on MS-COCO [16] using the DINO detector [34]. Across all settings, SparseSAM consistently achieves the best accuracy-efficiency trade-off, delivering the highest inference speed while preserving segmentation quality.

Attention Comparison. SpargeAttention [35] and PISA [14] are limited by SAM’s local attention design, where many attention heads operate on relatively small token maps (196×196). In this regime, the overhead of dynamic sparsification often outweighs the benefits of FlashAttention-style

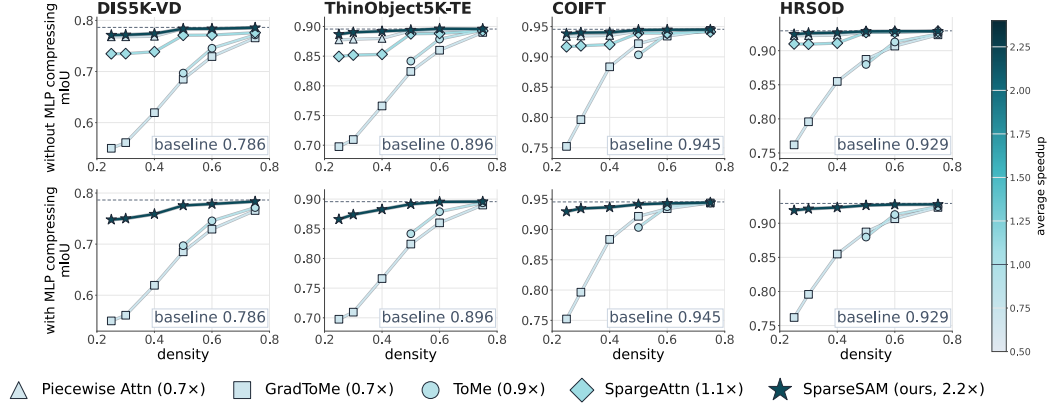


Figure 8: **Robustness Under Extreme Compression.** SparseSAM sets a new standard for efficient segmentation, consistently outperforming existing dynamic and merging-based approaches across density levels. Notably, at high compression rates (density < 0.4), where competing methods suffer catastrophic quality degradation, SparseSAM preserves near-baseline fidelity. This stability allows for a $2.2\times$ end-to-end acceleration without the typical trade-off between inference speed and mask quality.

kernels [5]. PISA introduces additional Top-K selection and Taylor approximation overhead, making it slower than the dense baseline in some cases (e.g., $0.73\times$ speedup on SAM-B at 25% density). In contrast, SparseSAM reuses a fixed Z-curve permutation across all encoder layers, avoiding costly runtime operations. As a result, it achieves the highest efficiency, reaching $1.63\times$ speedup on SAM-B at 25% density, compared to $1.25\times$ for SpargeAttention.

Joint Attention and MLP Compression. SparseSAM also outperforms token-merging approaches such as ToMe [1], which suffer from inaccurate merge-and-unmerge operations that degrade segmentation fidelity. At 50% density on SAM-L, ToMe achieves 0.425 mAP, while SparseSAM preserves 0.482 mAP and achieves up to $1.81\times$ speedup.

5.2.2 High Quality Segmentation

To better stress-test compression robustness, we evaluate SparseSAM on HQ-44K [12], which includes DIS5K-VD, ThinObject5K-TE, COIFT, and HRSOD, focusing on thin and small-object segmentation. As shown in Figure 8, SparseSAM consistently preserves higher segmentation fidelity than both dynamic masking and token-reduction baselines.

Attention-Only Analysis. Under heavy compression, our Stripe-Sort Attention remains highly stable. Leveraging deterministic Z-order interleaving and static A-shape masking, SparseSAM maintains performance close to the dense model even at 0.25 density (75% sparsity), avoiding the collapse observed in dynamic masking methods. For example, on COIFT at 0.3 density, SparseSAM achieves 0.945 mIoU, outperforming SpargeAttention [35] (0.919) and PISA [14] (0.937). A similar gain is observed on ThinObject5K-TE, where SparseSAM reaches 0.892 mIoU, exceeding both PISA (0.878) and SpargeAttention (0.850).

Joint Attention and MLP Compression. With residual-consistent MLP compression, SparseSAM offers a strong accuracy–efficiency trade-off. At 0.5 density, it achieves 0.782 mIoU on DIS5K-VD, while ToMe [1] (0.692) degrades substantially. Unlike token merging, our method preserves key information flow via residual propagation, allowing even low-rank tokens to contribute to later layers and preventing the information loss seen in merging-based approaches.

5.3 Ablation Studies

Finetune recovery. Despite being a training-free method by design, we also attempt to recover the performance of the model by fine-tuning only the MLP layers of SparseSAM for 15 minutes under low-density settings. As shown in Table 3, this simple adaptation consistently improves mIoU across datasets while keeping inference latency unchanged. At 25% density, mIoU increases from 74.79 to 76.92 on DIS5K-VD and from 91.79 to 93.01 on COIFT, with similar gains at 50% density. This

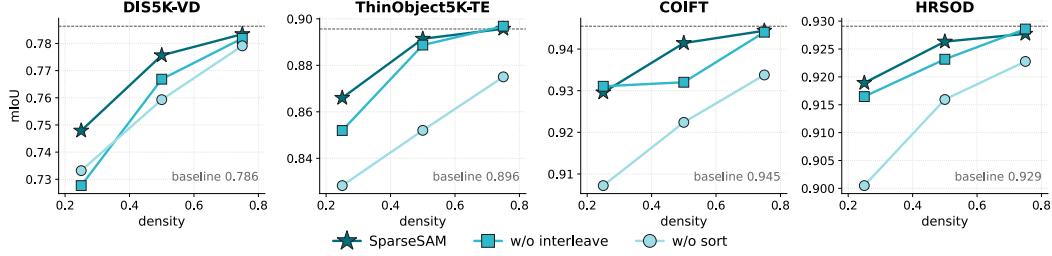


Figure 9: Performance breakdown with different permutation orders.

demonstrates that minimal MLP fine-tuning effectively recovers accuracy under aggressive token reduction without sacrificing speed.

Table 3: **Performance Recovery via Fine-tuning.** Fine-tuning SparseSAM at low densities (25% and 50%) significantly closes the mIoU gap while maintaining the same accelerated inference latency.

Method	Density	DIS5K-VD		COIFT		HRSOD	
		mIoU	Time (ms)	mIoU	Time (ms)	mIoU	Time (ms)
Base Model	100%	78.63	55.02	94.55	50.23	92.91	54.67
• SparseSAM	25%	74.79	25.97	91.79	24.90	91.29	26.69
• SparseSAM (+FT)	25%	76.92	25.97	93.01	24.90	91.96	26.69
• SparseSAM (finetuned)	50%	77.56	29.16	93.12	27.79	92.40	29.31
• SparseSAM (+FT)	50%	78.28	29.16	93.97	27.79	92.71	29.31

Different permutation orders in Sparse Attention kernel. Our token permutation combines (i) *Gradient-Based Ordering*, which prioritizes high-importance Z-curve groups, and (ii) *Stripe-Sort Permutation*, which interleaves groups to maintain spatially uniform kept tokens. We evaluate two ablations on SAM-HQ ViT-L over HQ44K: *w/o interleave*, which removes interleaving while preserving ranking, and *w/o sort*, which preserves interleaving but removes content-aware ranking. As shown in Figure 9, all variants perform similarly at mild compression ($r = 0.75$), but differences become significant at lower densities. At $r = 0.25$, removing interleaving reduces performance by up to -0.020 mIoU on DIS5K-VD, particularly for thin structures. Removing ranking causes larger drops, especially on ThinObject5K (-0.038) and COIFT (-0.022), indicating that content-aware prioritization is the primary factor under aggressive compression.

Can residual-consistency MLP be used on other segmentation backbones? To further confirm the generality of our residual-consistency MLP, we apply SparseSAM to SAM2, where the tokens passing through each MLP layer are routed by the same residual-consistency mechanism. As shown in Table 4, our observation transfers cleanly to the Hiera encoder of SAM2, whose backbone likewise

exhibits distinct token semantics across different regions of the image. We also attempted to adapt SparseSAM to other backbone that was train on contrastive learning objective and evaluate on Imagenet [6] for more information, please refer to A.5 and A.6.

6 Conclusion

We introduced SparseSAM, a training-free framework that accelerates the Segment Anything Model and its successors. By combining Stripe-Sort Permutation to exploit spatial redundancy with a Residual-Consistency MLP to preserve critical information flow, SparseSAM achieves a $2\times$ inference speedup and $2.8\times$ memory reduction with negligible fidelity loss. Validated across five diverse datasets, our approach maintains high performance while enabling the deployment of large-scale segmentation foundation models on resource-constrained edge devices.

Table 4: SparseSAM applied to SAM2 with the HQ-Hiera-L encoder.

r	COIFT		DIS5K-VD		HRSOD		ThinObject5K	
	mIoU	bIoU	mIoU	bIoU	mIoU	bIoU	mIoU	bIoU
1.00	0.951	0.908	0.753	0.670	0.909	0.854	0.903	0.819
0.70	0.951	0.907	0.754	0.674	0.912	0.858	0.906	0.825
0.50	0.951	0.906	0.761	0.677	0.916	0.861	0.913	0.834
0.30	0.946	0.901	0.767	0.678	0.925	0.867	0.912	0.824

For future work, several promising avenues remain. First, since our deterministic Z-order permutation is content-agnostic, exploring hybrid strategies could allow the model to dynamically prioritize semantic importance, better capturing long-range dependencies in complex scenes without losing the efficiency of static sparsity. Second, we aim to investigate layer-adaptive routing for the Residual-Consistency MLP to better account for evolving feature hierarchies as tokens transition from low-level textures to high-level representations. Finally, porting SparseSAM to backends like OpenCL or specialized DSP instructions will facilitate high-performance deployment across a wider array of mobile and edge devices.

Acknowledgments and Disclosure of Funding

This work was supported by the Siebel School of Computing and Data Science at the University of Illinois Urbana-Champaign (UIUC) and by the VinUni-Illinois Smart Health Center (VISHC), VinUniversity. The authors also thank the International Max Planck Research School for Intelligent Systems (IMPRS-IS) for providing feedback and additional computing resources.

References

- [1] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster. *arXiv preprint arXiv:2210.09461*, 2022.
- [2] Daniel Bolya, Po-Yao Huang, Peize Sun, Jang Hyun Cho, Andrea Madotto, Chen Wei, Tengyu Ma, Jiale Zhi, Jathushan Rajasegaran, Hanoona Rasheed, et al. Perception encoder: The best visual embeddings are not at the output of the network. *arXiv preprint arXiv:2504.13181*, 2025.
- [3] Zigeng Chen, Gongfan Fang, Xinyin Ma, and Xinchao Wang. Slimsam: 0.1% data makes segment anything slim. *Advances in Neural Information Processing Systems*, 37:39434–39461, 2024.
- [4] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [5] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [6] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- [7] Cihan Erkan and Selim Aksoy. Space-filling curves for modeling spatial context in transformer-based whole slide image classification. In *Medical Imaging 2023: Digital and Computational Pathology*, volume 12471, pages 416–423. SPIE, 2023.
- [8] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. YOLOX: Exceeding yolo series in 2021. *arXiv preprint arXiv:2107.08430*, 2021.
- [9] Rafael C Gonzalez. *Digital image processing*. Pearson education india, 2009.
- [10] Ding Jia, Yuhui Yuan, Haodi He, Xiaopei Wu, Haojun Yu, Weihong Lin, Lei Sun, Chao Zhang, and Han Hu. Detrs with hybrid matching. *arXiv preprint arXiv:2207.13080*, 2022.
- [11] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. *Advances in Neural Information Processing Systems*, 37:52481–52515, 2024.
- [12] Lei Ke, Mingqiao Ye, Martin Danelljan, Yifan Liu, Yu-Wing Tai, Chi-Keung Tang, and Fisher Yu. Segment anything in high quality. *arXiv preprint arXiv:2306.01567*, 2023.
- [13] Jiamei Kim, Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Marlin: FP16xINT4 LLM inference kernel that can achieve near-ideal 4x speedups up to medium batchsizes. <https://github.com/IST-DASLab/marlin>, 2023. Accessed: 2026-05-07.

- [14] Haopeng Li, Shitong Shao, Wenliang Zhong, Zikai Zhou, Lichen Bai, Hui Xiong, and Zeke Xie. Pisa: Piecewise sparse attention is wiser for efficient diffusion transformers. *arXiv preprint arXiv:2602.01077*, 2026.
- [15] Weicong Liang, Yuhui Yuan, Henghui Ding, Xiao Luo, Weihong Lin, Ding Jia, Zheng Zhang, Chao Zhang, and Han Hu. Expediting large-scale vision transformer for dense prediction without fine-tuning. In *Advances in Neural Information Processing Systems*, volume 35, pages 35462–35477, 2022.
- [16] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. *arXiv preprint arXiv:1405.0312*, 2014.
- [17] Liyang Liu, Shilong Zhang, Zhanghui Kuang, Aojun Zhou, Jing-Hao Xue, Xinjiang Wang, Yimin Chen, Wenming Yang, Qingmin Liao, and Wayne Zhang. Group fisher pruning for practical network compression. In *International Conference on Machine Learning*, pages 7021–7032. PMLR, 2021.
- [18] Yifan Liu, Ke Chen, Chris Liu, Zengchang Qin, Zhenbo Luo, and Jingdong Wang. Structured knowledge distillation for semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2604–2613, 2019.
- [19] Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. Learning efficient convolutional networks through network slimming. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2736–2744, 2017.
- [20] Chengtao Lv, Hong Chen, Jinyang Guo, Yifu Ding, and Xianglong Liu. Ptq4sam: Post-training quantization for segment anything. In *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition*, pages 15941–15951, 2024.
- [21] Guy M Morton. *A computer oriented geodetic data base and a new technique in file sequencing*. International Business Machines Company, 1966.
- [22] Duy MH Nguyen, Tuan A Tran, Duong Nguyen, Siwei Xie, Trung Q Nguyen, Mai TN Truong, Daniel Palenicek, An T Le, Michael Barz, TrungTin Nguyen, et al. Structsam: Structure-and spectrum-preserving token merging for segment anything models. *arXiv preprint arXiv:2603.07307*, 2026.
- [23] Navin Ranjan and Andreas Savakis. Mix-qsam: Mixed-precision quantization of the segment anything model. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 3280–3290, 2025.
- [24] Jens Stücker, Oliver Hahn, Lukas Winkler, Adrian Gutierrez Adame, and Thomas Flöss. Jz-tree: Gpu friendly neighbour search and friends-of-friends with dual tree walks in jax plus cuda. *arXiv preprint arXiv:2604.05885*, 2026.
- [25] Chau Tran, Duy MH Nguyen, Manh-Duy Nguyen, TrungTin Nguyen, Ngan Le, Pengtao Xie, Daniel Sonntag, James Y Zou, Binh Nguyen, and Mathias Niepert. Accelerating transformers with spectrum-preserving token merging. *Advances in Neural Information Processing Systems*, 37:30772–30810, 2024.
- [26] Tuan Anh Tran, Duy MH Nguyen, Hoai-Chau Tran, Michael Barz, Khoa D Doan, Roger Wattenhofer, Ngo Anh Vien, Mathias Niepert, Daniel Sonntag, and Paul Swoboda. How many tokens do 3d point cloud transformer architectures really need? *arXiv preprint arXiv:2511.05449*, 2025.
- [27] Changyuan Wang, Ziwei Wang, Xiuwei Xu, Yansong Tang, Jie Zhou, and Jiwen Lu. Q-vlm: Post-training quantization for large vision-language models. *Advances in Neural Information Processing Systems*, 37:114553–114573, 2024.
- [28] Haocheng Xi, Shuo Yang, Yilong Zhao, Chenfeng Xu, Muyang Li, Xiuyu Li, Yujun Lin, Han Cai, Jintao Zhang, Dacheng Li, et al. Sparse videogen: Accelerating video diffusion transformers with spatial-temporal sparsity. *arXiv preprint arXiv:2502.01776*, 2025.

- [29] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2024.
- [30] Yunyang Xiong, Bala Varadarajan, Lemeng Wu, Xiaoyu Xiang, Fanyi Xiao, Chenchen Zhu, Xiaoliang Dai, Dilin Wang, Fei Sun, Forrest Iandola, et al. EfficientSAM: Leveraged masked image pretraining for efficient segment anything. *arXiv preprint arXiv:2312.00863*, 2023.
- [31] Shuo Yang, Haocheng Xi, Yilong Zhao, Muyang Li, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Chenfeng Xu, Kelly Peng, et al. Sparse videogen2: Accelerate video generation with sparse attention via semantic-aware permutation. *arXiv preprint arXiv:2505.18875*, 2025.
- [32] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. Flashinfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005*, 2025.
- [33] Chaoning Zhang, Dongshen Han, Yu Qiao, Jung Uk Kim, Sung-Ho Bae, Seungkyu Lee, and Choong Seon Hong. Faster segment anything: Towards lightweight sam for mobile applications. *arXiv preprint arXiv:2306.14289*, 2023.
- [34] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. Dino: Detr with improved denoising anchor boxes for end-to-end object detection. *arXiv preprint arXiv:2203.03605*, 2022.
- [35] Jiarui Zhang, Chao Xiang, Haofeng Huang, Jingyan Wei, Haoyuan Xi, Jun Zhu, and Jianfei Chen. Spargeattention: Accurate and training-free sparse attention accelerating any model inference. *arXiv preprint arXiv:2502.18137*, 2025.
- [36] Wenlun Zhang, Yunshan Zhong, Shimpei Ando, and Kentaro Yoshioka. Ahcptq: Accurate and hardware-compatible post-training quantization for segment anything model. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22383–22392, 2025.
- [37] Xu Zhao, Wenchao Ding, Yongqi An, Yinglong Du, Tao Yu, Min Li, Ming Tang, and Jinqiao Wang. Fast segment anything. *arXiv preprint arXiv:2306.12156*, 2023.
- [38] Chong Zhou, Xiangtai Li, Chen Change Loy, and Bo Dai. Edgesam: Prompt-driven edge-aware segmentation with segment anything. *arXiv preprint*, 2023.

A Appendix

A.1 Attention Kernel Pseudo Code

Algorithm 1 A-Shape Windowed FlashAttention-2 with Decomposed 2D Rel-Pos Bias

```

// Attention input with decomposed 2D bias  $\mathbf{B}[q, k] = \mathbf{B}^H[q, k_{row}] + \mathbf{B}^W[q, k_{col}]$  on a  $w \times w$  grid
Input:  $\mathbf{Q} \in \mathbb{R}^{S_q \times d}$ ,  $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{S_k \times d}$ ;  $\mathbf{B}^H, \mathbf{B}^W \in \mathbb{R}^{S_q \times w}$ ,  $w = \sqrt{S_k}$ 
// Window-major  $\rightarrow$  spatial perms (for bias lookup); A-mask density ratio; softmax scale
 $\sigma_Q: [S_q] \rightarrow [S_q]$ ,  $\sigma_K: [S_k] \rightarrow [S_k]$ ;  $r \in [0, 1]$ ;  $\tau = 1/\sqrt{d}$ 
// Tile sizes, counts, slices, tile-local indices
Tiling:  $B_{row}, B_{col}$ ;  $T_{row} = \lceil S_q/B_{row} \rceil$ ,  $T_{col} = \lceil S_k/B_{col} \rceil$ ;  $\mathbf{Q}_i \triangleq \mathbf{Q}[iB_{row}:(i+1)B_{row}]$ ,  $\mathbf{K}_j, \mathbf{V}_j$ ;
 $row \in [0, B_{row})$ ,  $col \in [0, B_{col})$ 
Output:  $\mathbf{O} \in \mathbb{R}^{S_q \times d}$ 
// Active K-tiles: first  $\lfloor rT_{col} \rfloor$  columns plus the diagonal  $j=i$ 
 $\mathcal{J}_i \leftarrow \{0, \dots, \lfloor rT_{col} \rfloor - 1\} \cup \{i\}$ 
// Gather bias rows: query at tile-local row row has spatial position  $\sigma_Q(iB_{row}+row)$ 
 $\mathbf{B}_i^H[row, p] \leftarrow \mathbf{B}^H[\sigma_Q(iB_{row}+row), p]$ ;  $\mathbf{B}_i^W[row, p] \leftarrow \mathbf{B}^W[\sigma_Q(iB_{row}+row), p]$ 
// Online-softmax state: per-row max, denom, output accumulator
 $\mathbf{m} \leftarrow -\infty$ ,  $\ell \leftarrow 0$ ,  $\mathbf{O}_{acc} \leftarrow 0$ 
for  $j \in \mathcal{J}_i$  do
   $\mathbf{S} \leftarrow \mathbf{Q}_i \mathbf{K}_j^\top$ 
  // Add bias;  $\div \tau$  cancels the  $\tau$  in the softmax exponent below
   $\mathbf{S}[row, col] += (\mathbf{B}_i^H[row, \sigma_K(col) \div w] + \mathbf{B}_i^W[row, \sigma_K(col) \bmod w]) / \tau$ 
   $\mathbf{S}[row, col] \leftarrow -\infty$  where  $jB_{col} + col \geq S_k$ 
  // FA online-softmax update
   $\mathbf{m}' \leftarrow \max(\mathbf{m}, \text{rowmax } \mathbf{S})$ ;  $\tilde{\mathbf{P}} \leftarrow \exp(\tau(\mathbf{S} - \mathbf{m}'))$ ;  $\alpha \leftarrow \exp(\tau(\mathbf{m} - \mathbf{m}'))$ 
   $\ell \leftarrow \alpha \odot \ell + \text{rowsum } \tilde{\mathbf{P}}$ ;  $\mathbf{O}_{acc} \leftarrow \text{diag}(\alpha)\mathbf{O}_{acc} + \tilde{\mathbf{P}}\mathbf{V}_j$ ;  $\mathbf{m} \leftarrow \mathbf{m}'$ 
end for
 $\mathbf{O}_i \leftarrow \text{diag}(\ell)^{-1} \mathbf{O}_{acc}$ 

```

In SAM-based models, the attention operator includes a non-trivial relative positional bias that is directly added to the attention logits. To efficiently integrate a customized sparse attention kernel, we fuse this operation into a FlashAttention-2-style computation with decomposed 2D positional encoding, where the bias is factorized as $\mathbf{B}[q, k] = \mathbf{B}^H[q, k_{row}] + \mathbf{B}^W[q, k_{col}]$ on a $w \times w$ grid with $w = \sqrt{S_k}$.

The input features $\mathbf{Q} \in \mathbb{R}^{S_q \times d}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{S_k \times d}$ are first reordered using spatial permutations σ_Q and σ_K , which improve memory locality under a window-major tiling scheme. Attention is computed in a block-wise manner over tiles $\mathbf{Q}_i$ and $\mathbf{K}_j$, with tile sizes B_{row} and B_{col} , and only a subset of key blocks $\mathcal{J}_i$ is evaluated according to a structured A-shaped sparsity mask (controlled by density ratio r), which is statically compiled and incurs no runtime overhead.

For each tile interaction, attention scores are computed as $\mathbf{S} = \mathbf{Q}_i \mathbf{K}_j^\top$, and the decomposed positional bias is added via efficient index-based lookup using $\mathbf{B}^H$ and $\mathbf{B}^W$ after applying σ_Q and σ_K . Entries outside valid key boundaries are masked to $-\infty$.

The kernel follows the FlashAttention-2 online softmax formulation, maintaining running statistics $\mathbf{m}$ (row-wise maxima), ℓ (normalization terms), and $\mathbf{O}_{acc}$ (output accumulator). These are updated incrementally across key tiles to ensure numerical stability while streaming. The final output $\mathbf{O}_i$ is obtained by normalizing $\mathbf{O}_{acc}$ with ℓ after processing all active blocks in $\mathcal{J}_i$.

A.2 Full MS-COCO Results

Table 5 presents a detailed performance comparison of SparseSAM across the MS-COCO [16] dataset, utilizing SAM-B, SAM-L, and SAM-H backbones under three distinct detectors: DINO [34], H-DETR [10], and YOLOX [8]. The results indicate that SparseSAM consistently outperforms existing attention-only and token-reduction baselines in both segmentation accuracy (mAP) and inference efficiency. Notably, under joint attention and MLP compression, our method achieves

Table 5: MS-COCO results. Markers indicate compression type: $\circ$ attention-only and $\bullet$ attention+MLP.

Detector	Method	SAM-B			SAM-L			SAM-H		
		mAP	Density	Speedup	mAP	Density	Speedup	mAP	Density	Speedup
DINO	Base	0.468	100%	$\times 1.00$	0.495	100%	$\times 1.00$	0.499	100%	$\times 1.00$
	<i>Attention-only compression</i>									
	$\circ$ Sparge Attn	0.447	25%	$\times 1.25$	0.461	25%	$\times 1.23$	—	25%	—
		0.463	50%	$\times 1.24$	0.491	50%	$\times 1.21$	—	50%	—
	$\circ$ PieceWise Attn	0.455	25%	$\times 0.73$	0.483	25%	$\times 0.68$	0.486	25%	$\times 0.69$
		0.465	50%	$\times 0.72$	0.494	50%	$\times 0.67$	0.498	50%	$\times 0.65$
	$\circ$ SparseSAM (Ours)	0.459	25%	$\times 1.63$	0.487	25%	$\times 1.64$	0.494	25%	$\times 1.28$
		0.467	50%	$\times 1.55$	0.494	50%	$\times 1.57$	0.499	50%	$\times 1.22$
	<i>Attention + MLP compression</i>									
	$\bullet$ ToMe	0.403	50%	$\times 0.83$	0.425	50%	$\times 0.86$	0.430	50%	$\times 0.80$
		0.306	25%	$\times 0.86$	0.318	25%	$\times 0.89$	0.318	25%	$\times 0.94$
	$\bullet$ GradToMe	0.422	50%	$\times 0.71$	0.445	50%	$\times 0.74$	0.451	50%	$\times 0.79$
HDETR	$\bullet$ SparseSAM (Ours)	0.451	25%	$\times 2.15$	0.468	25%	$\times 2.05$	0.472	25%	$\times 1.59$
		0.459	50%	$\times 1.89$	0.482	50%	$\times 1.83$	0.487	50%	$\times 1.40$
	Base	0.402	100%	$\times 1.00$	0.424	100%	$\times 1.00$	0.427	100%	$\times 1.00$
	<i>Attention-only compression</i>									
	$\circ$ Sparge Attn	0.386	25%	$\times 1.25$	0.398	25%	$\times 1.20$	—	25%	—
		0.398	50%	$\times 1.24$	0.421	50%	$\times 1.18$	—	50%	—
	$\circ$ PieceWise Attn	0.391	25%	$\times 0.72$	0.415	25%	$\times 0.65$	0.418	25%	$\times 0.68$
		0.401	50%	$\times 0.70$	0.422	50%	$\times 0.63$	0.426	50%	$\times 0.64$
	$\circ$ SparseSAM (Ours)	0.396	25%	$\times 1.73$	0.418	25%	$\times 1.63$	0.423	25%	$\times 1.27$
		0.401	50%	$\times 1.66$	0.423	50%	$\times 1.56$	0.427	50%	$\times 1.20$
	<i>Attention + MLP compression</i>									
	$\bullet$ ToMe	0.351	50%	$\times 0.83$	0.369	50%	$\times 0.86$	0.373	50%	$\times 0.78$
		0.271	25%	$\times 0.86$	0.280	25%	$\times 0.88$	0.280	25%	$\times 0.93$
YOLOX	$\bullet$ GradToMe	0.366	50%	$\times 0.70$	0.386	50%	$\times 0.72$	0.390	50%	$\times 0.78$
		0.389	25%	$\times 2.16$	0.403	25%	$\times 2.05$	0.406	25%	$\times 1.58$
	$\bullet$ SparseSAM (Ours)	0.394	50%	$\times 1.91$	0.415	50%	$\times 1.83$	0.417	50%	$\times 1.37$
	Base	0.390	100%	$\times 1.00$	0.412	100%	$\times 1.00$	0.416	100%	$\times 1.00$
	<i>Attention-only compression</i>									
	$\circ$ Sparge Attn	0.374	25%	$\times 1.25$	0.386	25%	$\times 1.20$	—	25%	—
		0.386	50%	$\times 1.24$	0.409	50%	$\times 1.18$	—	50%	—
	$\circ$ PieceWise Attn	0.379	25%	$\times 0.71$	0.403	25%	$\times 0.65$	0.406	25%	$\times 0.67$
		0.388	50%	$\times 0.69$	0.410	50%	$\times 0.63$	0.414	50%	$\times 0.63$
	$\circ$ SparseSAM (Ours)	0.384	25%	$\times 1.70$	0.406	25%	$\times 1.63$	0.411	25%	$\times 1.29$
		0.390	50%	$\times 1.63$	0.410	50%	$\times 1.56$	0.415	50%	$\times 1.23$
	<i>Attention + MLP compression</i>									
	$\bullet$ ToMe	0.343	50%	$\times 0.83$	0.361	50%	$\times 0.86$	0.365	50%	$\times 0.79$
		0.267	25%	$\times 0.85$	0.276	25%	$\times 0.88$	0.276	25%	$\times 0.92$
	$\bullet$ GradToMe	0.356	50%	$\times 0.69$	0.377	50%	$\times 0.72$	0.381	50%	$\times 0.77$
		0.377	25%	$\times 2.15$	0.391	25%	$\times 2.05$	0.395	25%	$\times 1.58$
	$\bullet$ SparseSAM (Ours)	0.384	50%	$\times 1.92$	0.403	50%	$\times 1.83$	0.405	50%	$\times 1.40$

state-of-the-art speedups—reaching up to $2.16\times$ at 25% density—while maintaining mAP scores remarkably close to the dense base models. This demonstrates the robustness of SparseSAM’s structured sparsity across various model scales and detection frameworks, significantly mitigating the drastic accuracy drops observed in merging-based approaches like ToMe [1].

Table 6: Segmentation results. Markers indicate compression type: $\circ$ attention-only, $\bullet$ attention+MLP, and $\bullet$ denotes finetuned models.

Method	Density	DIS5K-VD		COIFT		ThinObject5K-TE		HRSOD	
		mIoU	Speedup	mIoU	Speedup	mIoU	Speedup	mIoU	Speedup
Base Model	100%	78.63	$\times 1.00$	94.55	$\times 1.00$	89.56	$\times 1.00$	92.91	$\times 1.00$
<i>Attention-only Sparsify</i>									
$\circ$ SpargeAttn	25%	73.48	$\times 1.26$	91.67	$\times 1.25$	84.94	$\times 1.24$	90.99	$\times 1.25$
	50%	77.09	$\times 1.24$	93.92	$\times 1.23$	88.76	$\times 1.25$	92.68	$\times 1.24$
	75%	77.61	$\times 1.16$	94.12	$\times 1.14$	89.26	$\times 1.15$	92.84	$\times 1.15$
$\circ$ PieceWise Attention	25%	76.79	$\times 0.71$	93.37	$\times 0.70$	87.74	$\times 0.72$	92.18	$\times 0.70$
	50%	78.38	$\times 0.69$	94.25	$\times 0.71$	89.26	$\times 0.70$	92.73	$\times 0.71$
	75%	78.54	$\times 0.68$	94.51	$\times 0.67$	89.49	$\times 0.67$	92.87	$\times 0.66$
$\circ$ SparseSAM (Ours)	25%	77.13	$\times 1.75$	93.83	$\times 1.63$	88.68	$\times 1.68$	92.41	$\times 1.71$
	50%	78.38	$\times 1.67$	94.44	$\times 1.56$	89.40	$\times 1.61$	92.83	$\times 1.63$
	75%	78.58	$\times 1.61$	94.49	$\times 1.50$	89.58	$\times 1.55$	92.86	$\times 1.58$
<i>Attention + MLP compression</i>									
$\bullet$ ToMe	50%	69.69	$\times 0.81$	90.33	$\times 0.81$	84.17	$\times 0.81$	87.98	$\times 0.84$
	75%	77.16	$\times 0.52$	94.36	$\times 0.51$	89.38	$\times 0.52$	92.43	$\times 0.51$
$\bullet$ GradToMe	25%	54.97	$\times 0.74$	75.22	$\times 0.75$	69.76	$\times 0.75$	76.19	$\times 0.77$
	50%	68.50	$\times 0.63$	92.17	$\times 0.64$	82.42	$\times 0.64$	88.73	$\times 0.63$
	75%	76.58	$\times 0.53$	94.37	$\times 0.54$	89.00	$\times 0.55$	92.32	$\times 0.54$
$\bullet$ SparseSAM (Ours)	25%	74.79	$\times 2.12$	91.79	$\times 2.02$	85.95	$\times 2.08$	91.29	$\times 2.05$
	50%	77.56	$\times 1.89$	93.12	$\times 1.81$	87.82	$\times 1.88$	92.40	$\times 1.87$
	75%	78.43	$\times 1.73$	94.12	$\times 1.64$	89.21	$\times 1.70$	92.62	$\times 1.71$
<i>Finetuning after Joint Compression</i>									
$\bullet$ SparseSAM (Ours)	25%	76.92	$\times 2.12$	93.01	$\times 2.02$	88.05	$\times 2.08$	91.96	$\times 2.05$
$\bullet$ SparseSAM (Ours)	50%	78.28	$\times 1.89$	93.97	$\times 1.81$	89.41	$\times 1.88$	92.71	$\times 1.87$

A.3 Full HQ44k Results

Table 6 presents an extensive evaluation of SparseSAM’s performance on zero-shot object segmentation across four challenging datasets including DIS5K-VD, COIFT, ThinObject5K-TE, and HRSOD from HQ-44K [12]. The results demonstrate that SparseSAM maintains high segmentation accuracy while achieving superior speedup compared to existing training-free compression methods like SpargeAttention [35] and PISA [14]. Under joint attention and MLP compression at 25% density, the model reaches peak speedup factors exceeding $2.0\times$ while suffering significantly less accuracy degradation than competitive methods such as ToMe [1]. The performance on ThinObject5K-TE particularly underscores the method’s ability to preserve intricate spatial details even at low density levels.

The final tier of the evaluation (blue markers) demonstrates that a lightweight finetuning stage successfully bridges the performance gap inherent in high-compression regimes. By allowing the model to adapt to the structured sparsity patterns in both the attention and MLP layers, SparseSAM recovers mIoU to near-base levels while fully preserving its throughput advantages. Specifically, the finetuned configurations at 25% and 50% density show negligible performance degradation compared to the 100% density baseline. This underscores the efficacy of our approach in delivering a robust, hardware-efficient solution that maintains high-fidelity segmentation without sacrificing state-of-the-art inference acceleration.

A.4 Hardware-Aware Performance Analysis

Table 7 provides a detailed throughput speedup analysis of SparseSAM, comparing the performance of attention-only compression against joint attention and MLP compression across A100 and RTX 3090 GPUs. The results reveal that incorporating our Residual-Preserving MLP technique consistently

Table 7: Throughput speedup comparison on different hardware and sparsity levels. We evaluate SparseSAM under two configurations: attention-only compression and joint attention + MLP compression.

Batchsize	GPU	25% Sparsity		50% Sparsity		75% Sparsity	
		Attn. + MLP	Attn. Only	Attn. + MLP	Attn. Only	Attn. + MLP	Attn. Only
1	A100	$\times 1.91$	$\times 1.65$	$\times 1.75$	$\times 1.56$	$\times 1.33$	$\times 1.23$
	RTX 3090	$\times 1.42$	$\times 1.32$	$\times 1.35$	$\times 1.25$	$\times 1.17$	$\times 1.15$
2	A100	$\times 1.95$	$\times 1.62$	$\times 1.75$	$\times 1.55$	$\times 1.33$	$\times 1.22$
	RTX 3090	$\times 1.45$	$\times 1.33$	$\times 1.36$	$\times 1.25$	$\times 1.16$	$\times 1.16$
4	A100	$\times 2.00$	$\times 1.62$	$\times 1.80$	$\times 1.55$	$\times 1.46$	$\times 1.42$
	RTX 3090	$\times 1.53$	$\times 1.33$	$\times 1.39$	$\times 1.26$	$\times 1.23$	$\times 1.27$
8	A100	$\times 2.05$	$\times 1.64$	$\times 1.83$	$\times 1.57$	$\times 1.66$	$\times 1.50$
	RTX 3090	$\times 1.57$	$\times 1.34$	$\times 1.45$	$\times 1.27$	$\times 1.28$	$\times 1.28$

enhances efficiency, yielding superior throughput compared to attention-only masking across all tested density ratios. Notably, the A100 architecture achieves a peak speedup of $2.05\times$ at 25% sparsity, demonstrating higher hardware utilization than the RTX 3090. Furthermore, the speedup gains for the joint compression configuration scale positively with larger batch sizes, validating that our structured sparsity approach effectively maximizes computational throughput while maintaining the benefits of training-free acceleration.

A.5 MLP update behavior in other global task backbones

To understand how MLPs distribute work across an encoder, we visualize the per-token MLP update norm $\|\Delta_{\text{MLP}}\|_2 = \|\text{MLP}(\text{LN}(x))\|_2$ at four representative blocks of Perception Encoder [2] (PE-Core-L14-336) and SAM-L on the same image. From Figure 10, it can be seen that the two backbones use their MLPs in markedly different ways. At block 0, behavior is broadly aligned: both models drive larger updates on the butterfly than on the background, indicating early object-aware encoding. Beyond the first block, the patterns diverge. PE, trained with an image-level contrastive objective, gradually loses spatial selectivity — by mid- and late-encoder, MLP updates are roughly uniform across all tokens, consistent with the network preparing a globally pooled embedding. SAM, trained for dense prediction, retains spatial selectivity throughout: foreground tokens consistently receive distinctive MLP updates, with the butterfly outline still clearly visible at block 23.

As expected from this observation, residual-consistency MLP fails catastrophically when applied to PE backbones, but performs surprisingly well on segmentation models.

A.6 Imagenet Results

Based on the observation in the previous section, we retain the Stripe-Sort Attention mechanism while replacing the MLP compression approach with a more traditional method.token merging used in [1]. Figure 11 compares Top-1 accuracy across different sparsity densities under two settings: without MLP compression (left) and with MLP compression (right). In both cases, SparseSAM consistently outperforms token-merging baselines (ToMe) and sparse attention (SpargAttn) across all density levels, maintaining performance close to or above the dense baseline (0.835). Without MLP compression, SparseSAM remains highly stable even at low densities (0.25–0.4), where competing methods show noticeable degradation. As density increases, all methods converge toward the baseline, but SparseSAM consistently stays ahead. When MLP compression is enabled, SparseSAM retains nearly identical accuracy trends while achieving higher efficiency ($1.4\times$ speedup). Notably, even under stronger compression, it preserves robustness across all density regimes, whereas baselines remain significantly below the dense reference, especially at low densities.

Overall, the results highlight that SparseSAM’s stripe-sort attention mechanism maintains a better accuracy-efficiency trade-off, with minimal degradation under joint attention and MLP compression.

A.7 More segment output comparisons

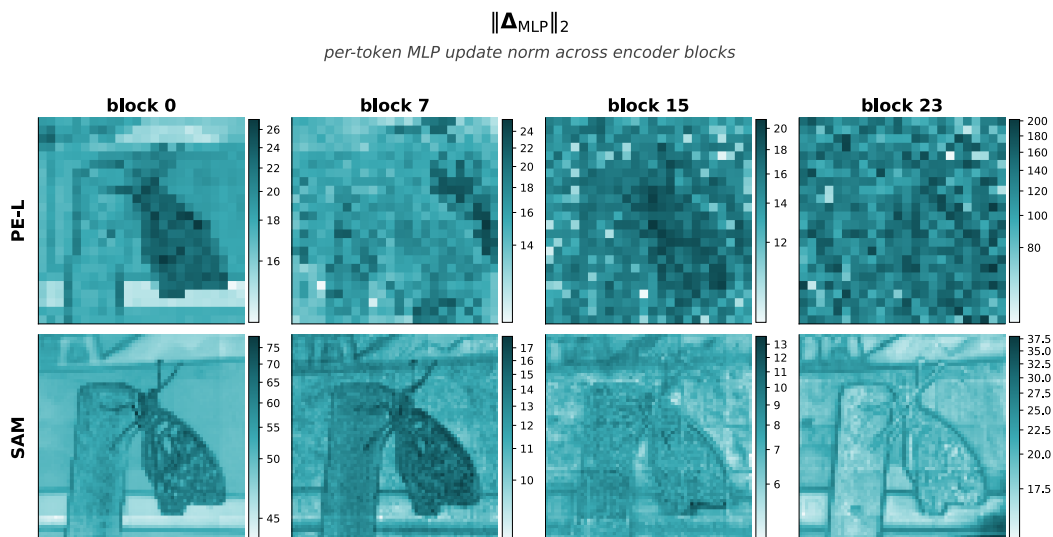


Figure 10: Per-token MLP update norm $\|\Delta_{\text{MLP}}\|_2$ at blocks 0/7/15/23 of PE-Core-L14-336 (top) and SAM-HQ ViT-L (bottom) on the same input. Reflecting their different pretraining inductive biases, the two backbones distribute MLP work in opposite ways. The first block looks similar across models — both focus updates on the butterfly — but in later layers, PE updates tokens uniformly across the image, while SAM stays spatially selective and keeps its MLP work concentrated on the foreground.

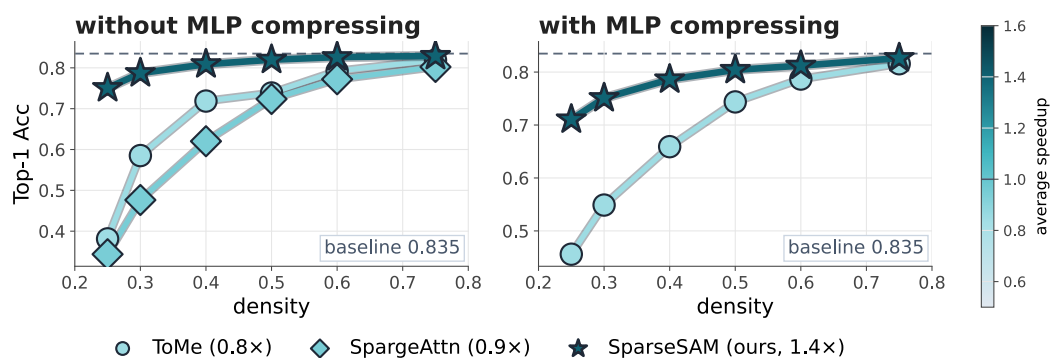


Figure 11: Results on ImageNet[6]

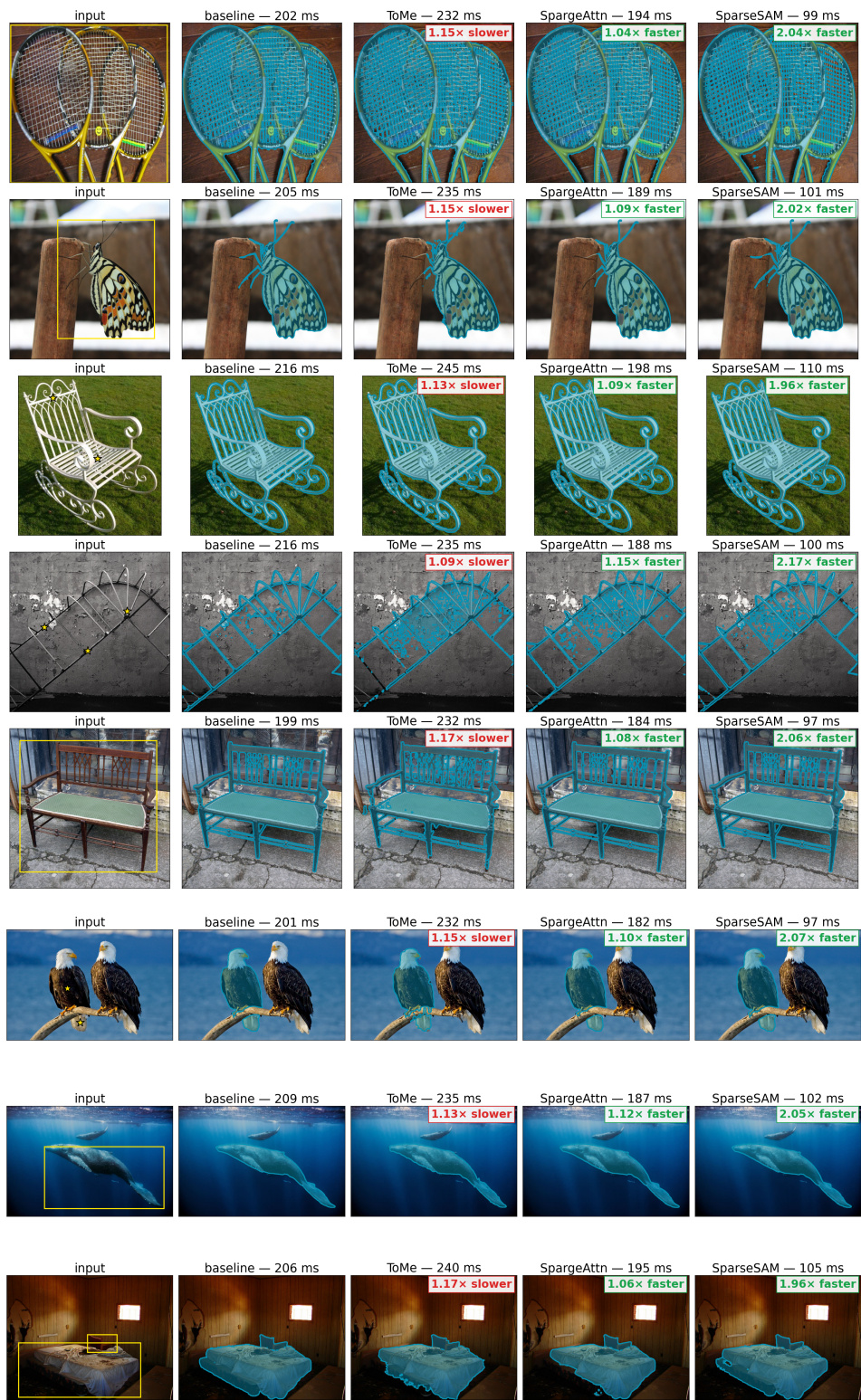


Figure 12: More output visualization